Introduction - F1Tenth



Contents

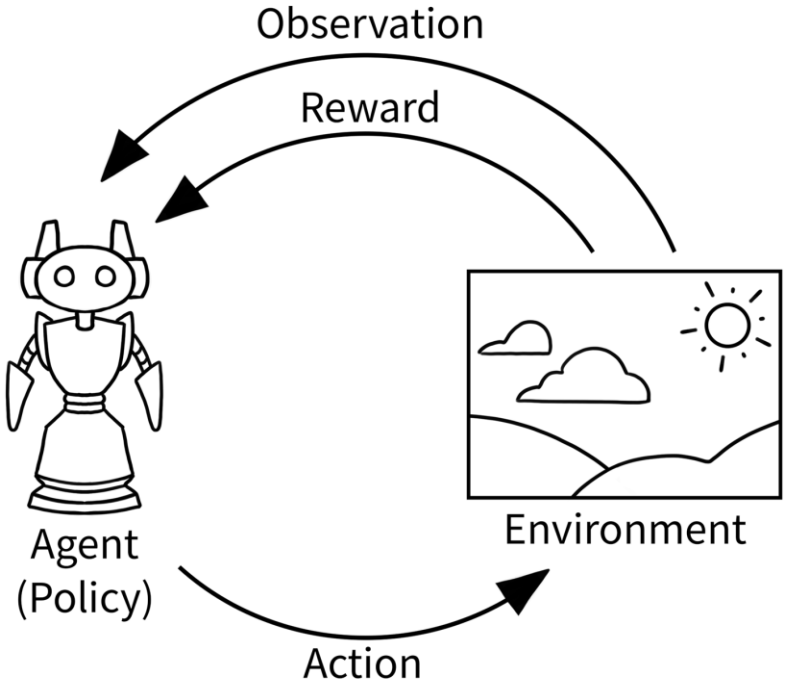
RLLAB
http://rllab.snu.ac.kr

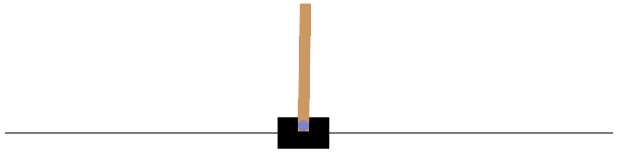
- **Basic Structure**
  - Workflow
  - Wrapper
  - Vectorized Env (opt)
  - Spaces (opt)
- **F1Tenth Gymnasium**
  - Descriptions
  - Example video
    - Simulation
    - Real-world

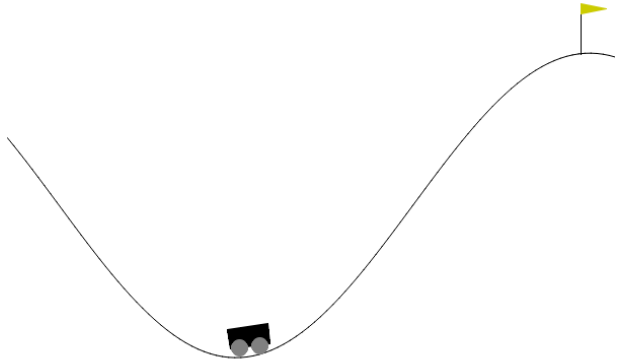
OpenAI Gymnasium

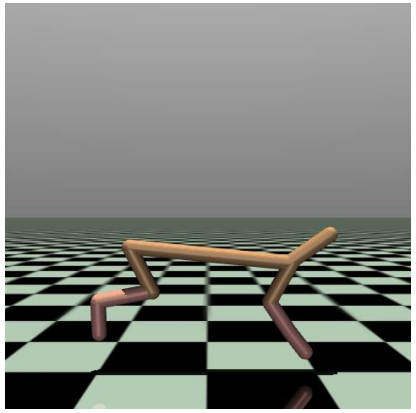
RLLAB
http://rllab.snu.ac.kr

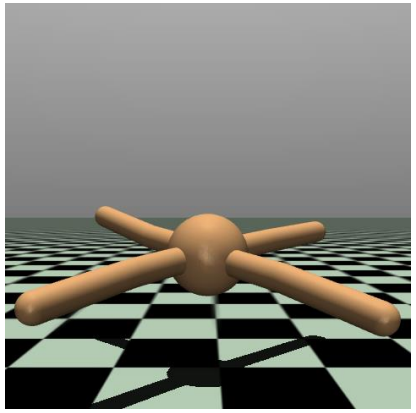
- OpenAI Gymnasium
  https://github.com/Farama-Foundation/Gymnasium

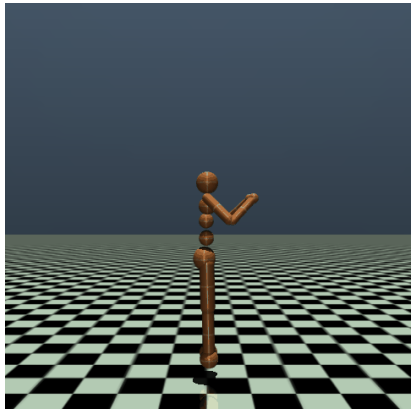












OpenAI Gymnasium

RLLAB
http://rllab.snu.ac.kr

Workflow

1. Define the environment and initialize it.

```
import gymnasium as gym

env = gym.make("LunarLander-v3", render_mode="human")
observation, info = env.reset()
```

- reset() : initialize the environment and get { *first observation* }
- step(action) : interact with the environment and get { *next observation*, *reward*, *done*, *info* }
- render() : visualize the environment
- close() : close the environment

OpenAI Gymnasium

2. Extract the action from the agent and interacts with the environment.

```
action = agent.get_action(obs)
next_obs, reward, terminated, truncated, info = env.step(action)
```

3. Repeat it until the episode ends.

```
# play one episode
while not done:
    action = agent.get_action(obs)
    next_obs, reward, terminated, truncated, info = env.step(action)

    # update the agent
    agent.update(obs, action, reward, terminated, next_obs)

    # update if the environment is done and the current obs
    done = terminated or truncated
    obs = next_obs
```

0 (False): continue episode
1 (True) : the terminal state (e.g. success, crash …)

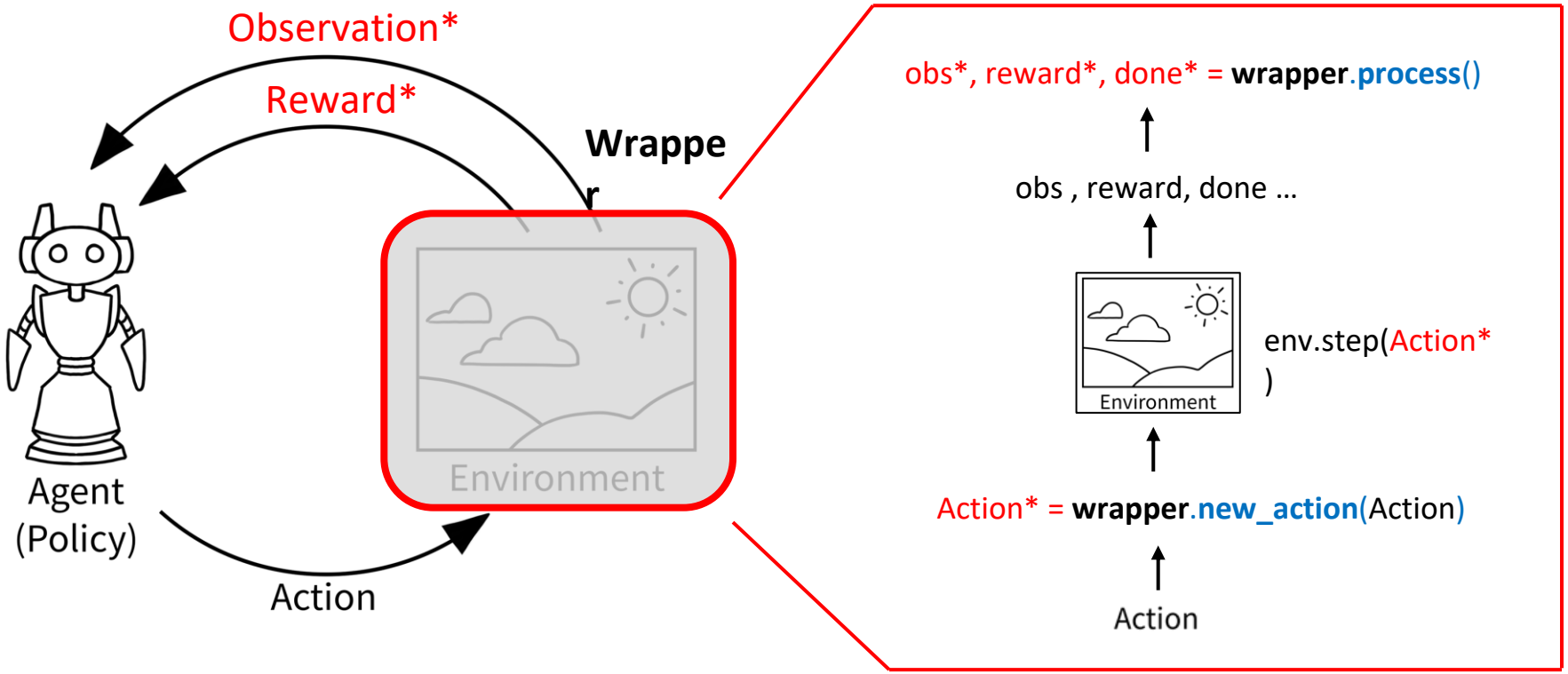
0 (False): step < *max_episode_steps*
1 (True) : time over

done = terminated **or** truncated
fail = terminated **and** (*not* truncated)

OpenAI Gymnasium

RLLAB
http://rllab.snu.ac.kr

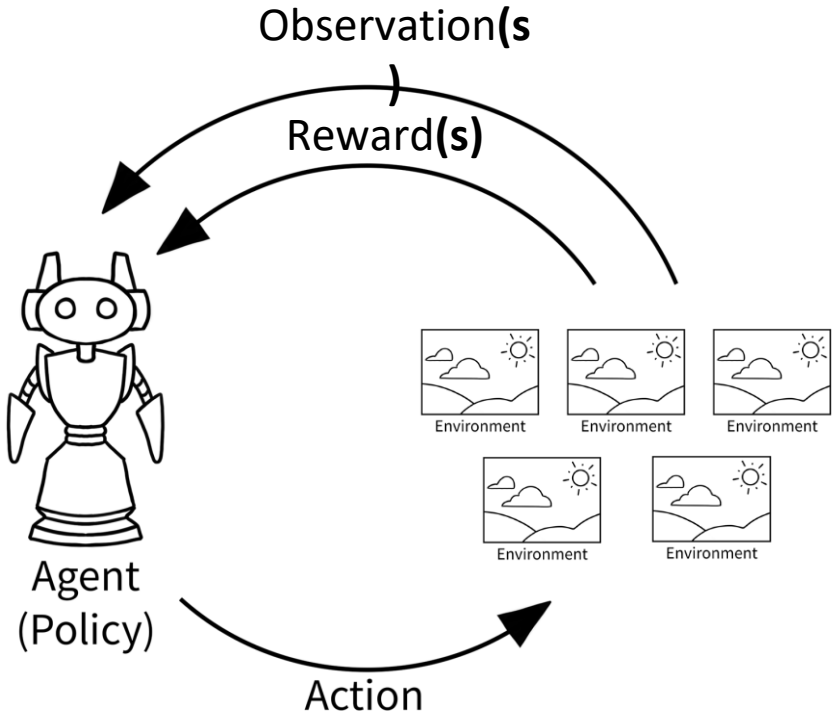
Wrapper



OpenAI Gymnasium (optional)

RLLAB
http://rllab.snu.ac.kr

Vectorized env



AsyncVectorEnv

```
class gymnasium.vector.AsyncVectorEnv(env_fns: Sequence[Callable[[], Env]], shared_memory: bool
    = True, copy: bool = True, context: str | None = None, daemon: bool = True, worker:
    Callable[[int, Callable[[], Env], Connection, Connection, bool, Queue], None] | None =
    None, observation_mode: str | Space = 'same')                                    [source]
```

ex)

```
envs = gym.vector.AsyncVectorEnv([
    lambda: gym.make("Pendulum-v1", g=9.81),
    lambda: gym.make("Pendulum-v1", g=1.62) ])
```

- trained in **diverse** dynamics simultaneously
- training time **reduces**

OpenAI Gymnasium (optional)

gymnasium.spaces

- observation_space : define the dimension of the observation (also contents)
- action_space : define the dimension of the action

```
self.action_space = spaces.Discrete(3)
self.observation_space = spaces.Box(0, 1, shape=(2,))
```

→ action = 0 or 1 or 2 (discrete)

→ obs = [0~1, 0~1] (e.g. [0.3, 0.7])

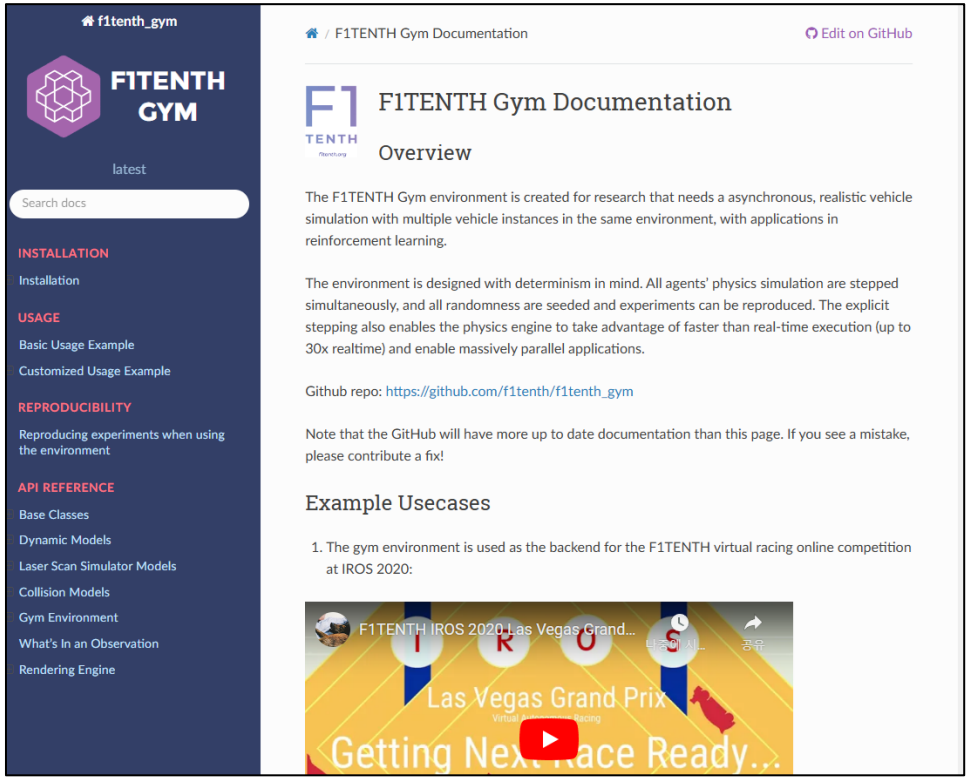
- `MultiBinary` - Supports single or matrices of binary values, used for holding down a button or if an agent has an object
- `MultiDiscrete` - Supports multiple discrete values with multiple axes, used for controller actions
- `Text` - Supports strings, used for passing agent messages, mission details, etc

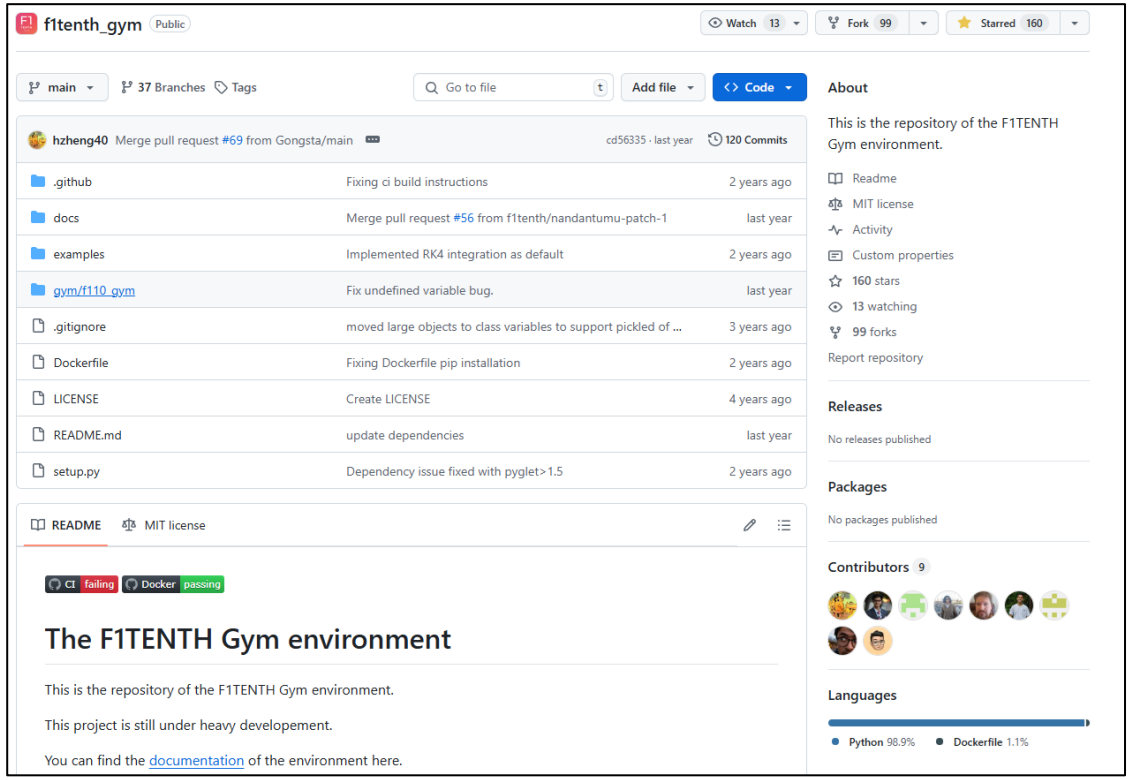
It's okay even if you don't understand!

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

Originated from : https://github.com/f1tenth/f1tenth_gym/tree/main
Documentation : https://f1tenth-gym.readthedocs.io/en/latest/





Observation	raw lidar values -> (1080,)
Action	[steering angle, target speed]

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

*** indicates the codes you should implement**

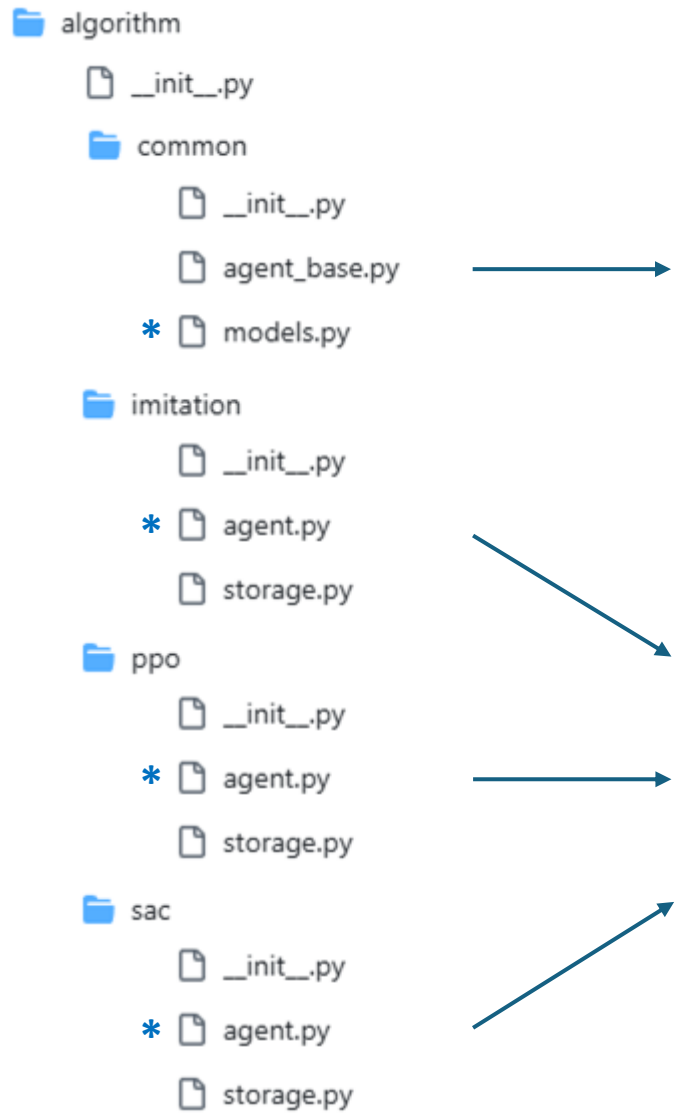
```
- f1tenth
  ├── * algorithm
  ├── configs
  ├── installation
  ├── leaderboard.py
  ├── main_il.py
  ├── main_rl.py
  ├── README.md
  ├── requirements.txt
  ├── * utils
  └── visualize.py
```

- ***algorithm** PPO, SAC, imitation learning codes
- ***configs** hyperparameter for algorithms and environment
- installation original f1tenth gymnasium
- *main_il/main_rl.py* python execution code (training)
- *leaderboard.py* python execution code (test)
- README.md guidance to use this repository
- ***utils** functional codes + ***wrapper**
- requirements necessary package list
- *visualize.py* python execution code (analysis)

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```



*** indicates the codes you should implement**

<class inheritance>

```python
class AgentBase(ABC):
    def __init__(self, args)

    def getAction(self, state:np.ndarray, deterministic:bool = True)
```

```python
class Agent(AgentBase):
    def __init__(self, args):
        super().__init__(args)

    def step(self, states, actions)

    def train(self)

    def save(self, model_num=None, log=True)

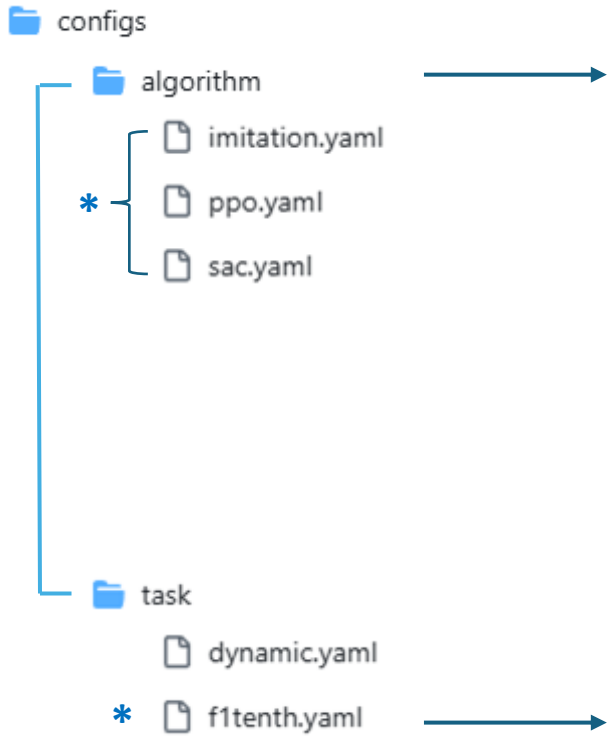
    def load(self, model_num=None)
```

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

*** indicates the codes you should implement**

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```




```yaml
# for network parameters
activation: LeakyReLU
hidden_dim: 256
log_std_init: -1.0
log_std_grad: false

# for training
train_frequency: 10000
minibatch_size: 128
discount_factor: 0.99
lr: 3.0e-4
```

```yaml
# base
map: null
max_episode_steps: 1000
obs_dim: 1080
action_dim: 2
show_centerline: false

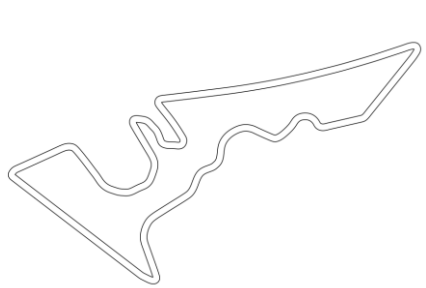
# env configs
timestep: 0.025
min_speed: 1.0
max_speed: 4.0
max_steer: 0.48    # calibration
```

F1Tenth Gymnasium

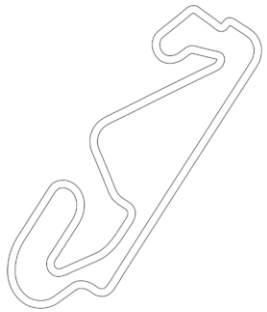
RLLAB
http://rllab.snu.ac.kr

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```

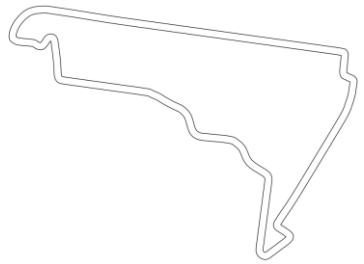
- installation
  - docs
  - examples
    - random_trackgen.py
    - run_in_circle.py → To align with real RC car dynamics
    - test.csv
    - waypoint_follow.py → Example code
  - f1tenth_gym
  - maps
  - tests
  - Dockerfile
  - LICENSE
  - README.md
  - poetry.lock
  - pyproject.toml
  - setup.cfg



Austin



Cataluny
a



Mexico City

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```

```python
while total_step < args.total_steps:

    # ======= collect trajectories ======= #
    start_time = time.time()
    step = 0
    observations, info = vec_env.reset()
    reward_history = [[] for _ in range(args.n_envs)]
    while step < args.n_steps:
        step += args.n_envs
        total_step += args.n_envs

        with torch.no_grad():
            clipped_action_tensor = agent.getAction(observations, deterministic=False)
            clipped_actions = clipped_action_tensor.detach().cpu().numpy()
            observations, rewards, terminates, truncates, infos = vec_env.step(clipped_actions)

        temp_fails = []
        temp_dones = []
        temp_observations = []
```

Do not change the execution code because of sim-to-real process !

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```

The F1TENTH Gym environment

This is the repository of the F1TENTH Gym environment.

You can find the original documentation of the environment here.

> Original author: *Hongrui Zheng*

> Author: *Hyeokjin Kwon, Geunje Cheon, Junseok Kim*

Quickstart

We recommend installing the simulation inside a virtualenv. You can install the environment by running:

```
cd [your-virtualenv-folder]
virtualenv F1tenth --python=python3.8
source [your-virtualenv-folder]/F1tenth/bin/activate
# pip install poetry
pip install --upgrade pip

cd [your-workspace-folder]
git clone https://github.com/ineogi2/f1tenth
cd installation
pip install -e .
cd [your-workspace-folder]
pip install -r requirements.txt
```

Then you can run a quick waypoint follow example by:

```
cd installation/examples
python3 waypoint_follow.py --render --map Spielberg
```

Calibration

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```

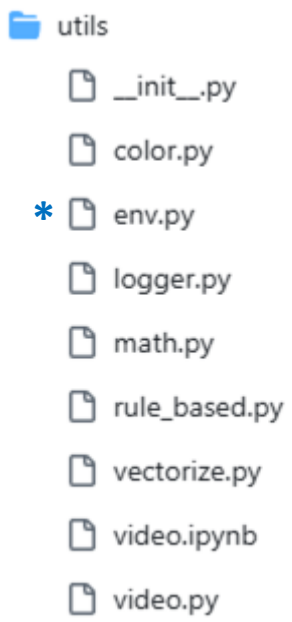
1. wandb — online logging tool *(optional but strongly recommended)*
2. ruamel.yaml — read the *yaml* type file
3. moviepy — save the rendered video
4. pandas — data processing
5. easydict — one of the package that helps to use dictionary

F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

*** indicates the codes you should implement**

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```




```python
class F1Wrapper(gym.Wrapper):
    def __init__(self, args, maps, render_mode=None) -> None:

        self._env = gym.make("f1tenth_gym:f1tenth-v0",
                             args=args,
                             maps=maps,
                             render_mode=render_mode)
        super().__init__(self._env)


    # ============================ implement here ============================ #
    # TODO
    # You need to implement your own reward function.
    def calc_reward(self):
    # ======================================================================== #



    # ============================ implement here ============================ #
    # TODO
    # You need to implement your own observation.
    def getObs(self, obs_dict, reset=False):
    # ======================================================================== #
```

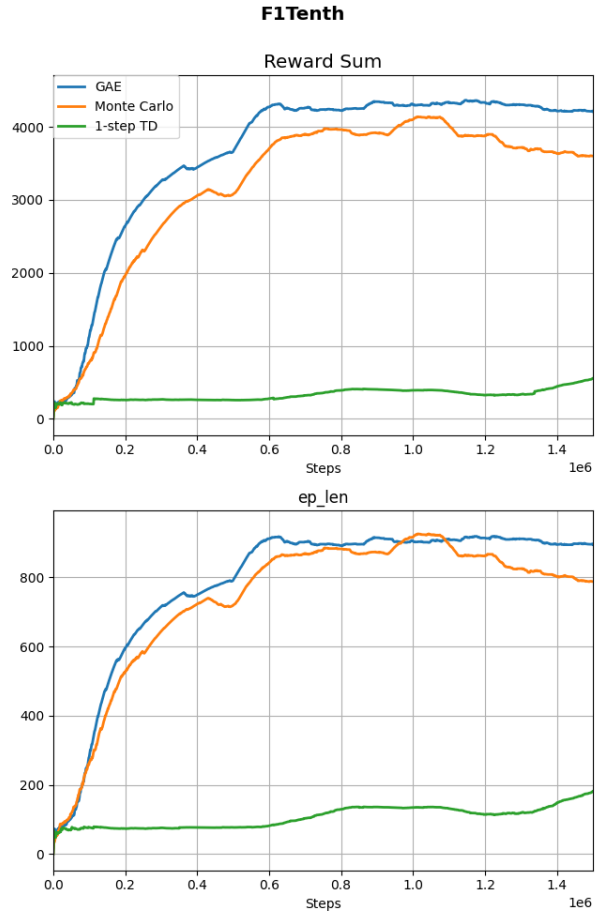
F1Tenth Gymnasium

RLLAB
http://rllab.snu.ac.kr

```
f1tenth
├── * algorithm
├── configs
├── installation
├── leaderboard.py
├── main_il.py
├── main_rl.py
├── README.md
├── requirements.txt
├── * utils
└── visualize.py
```

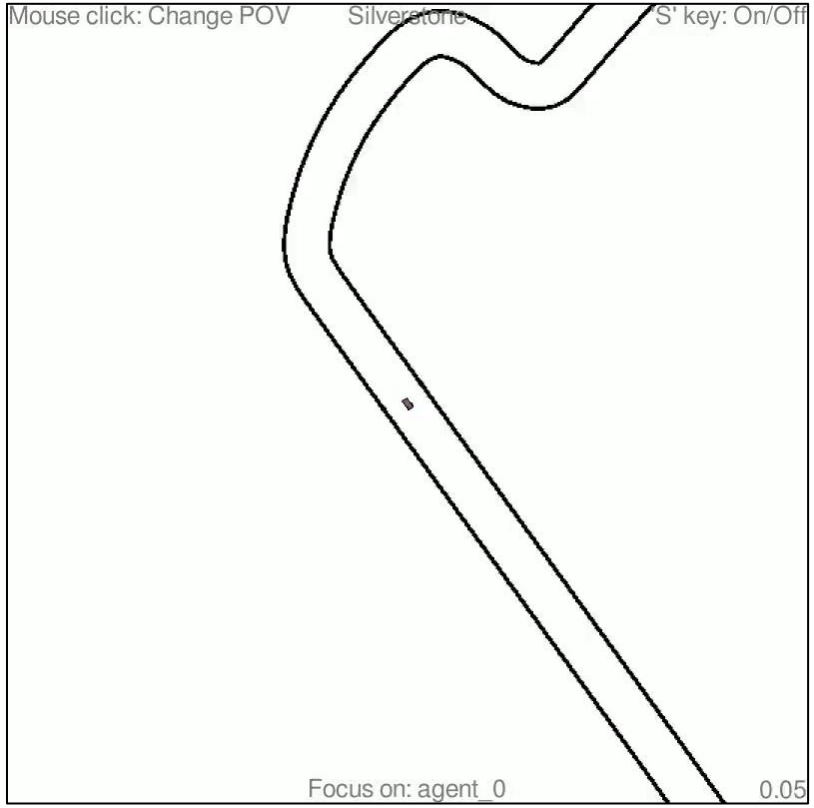
```python
def main():
    fig_size = 5
    window_size = 500
    interp_steps = 1000
    steps = int(1.5e6)
    env_name = "F1Tenth"
    algo_list = []
```

```python
def draw(env_name, item_list, algo_list, fig_size, window_size, interp_steps, is_horizon=True, steps=int(1e6)):
    if is_horizon:
        fig, ax_list = plt.subplots(nrows=1, ncols=len(item_list), figsize=(fig_size*len(item_list), fig_size))
    else:
        fig, ax_list = plt.subplots(nrows=len(item_list), ncols=1, figsize=(fig_size*1.3, fig_size*len(item_list)))
    if len(item_list) == 1:
        ax_list = [ax_list]
    if is_horizon == True:
        plt.suptitle(f'{env_name}', fontsize="x-large", style='bold')
    elif is_horizon == False:
        plt.suptitle(f'{env_name}\n', fontsize="x-large", fontweight='bold')
```

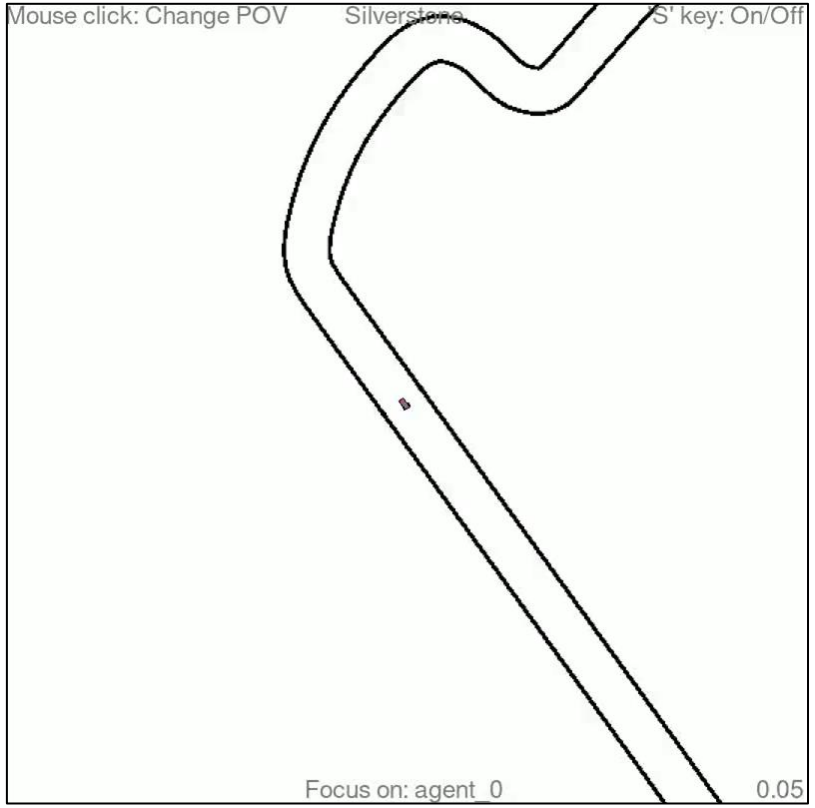


F1Tenth Gymnasium (sim)

RLLAB
http://rllab.snu.ac.kr



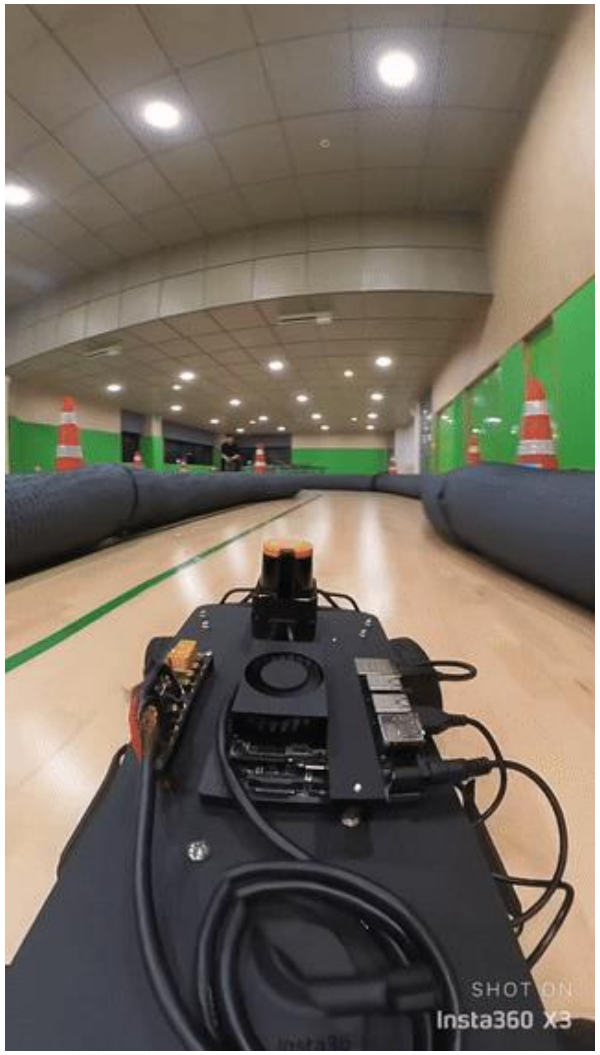
PPO



SAC

F1Tenth Gymnasium (real)

RLLAB
http://rllab.snu.ac.kr



Good Luck!